Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😁

alkemy

Clase grabada

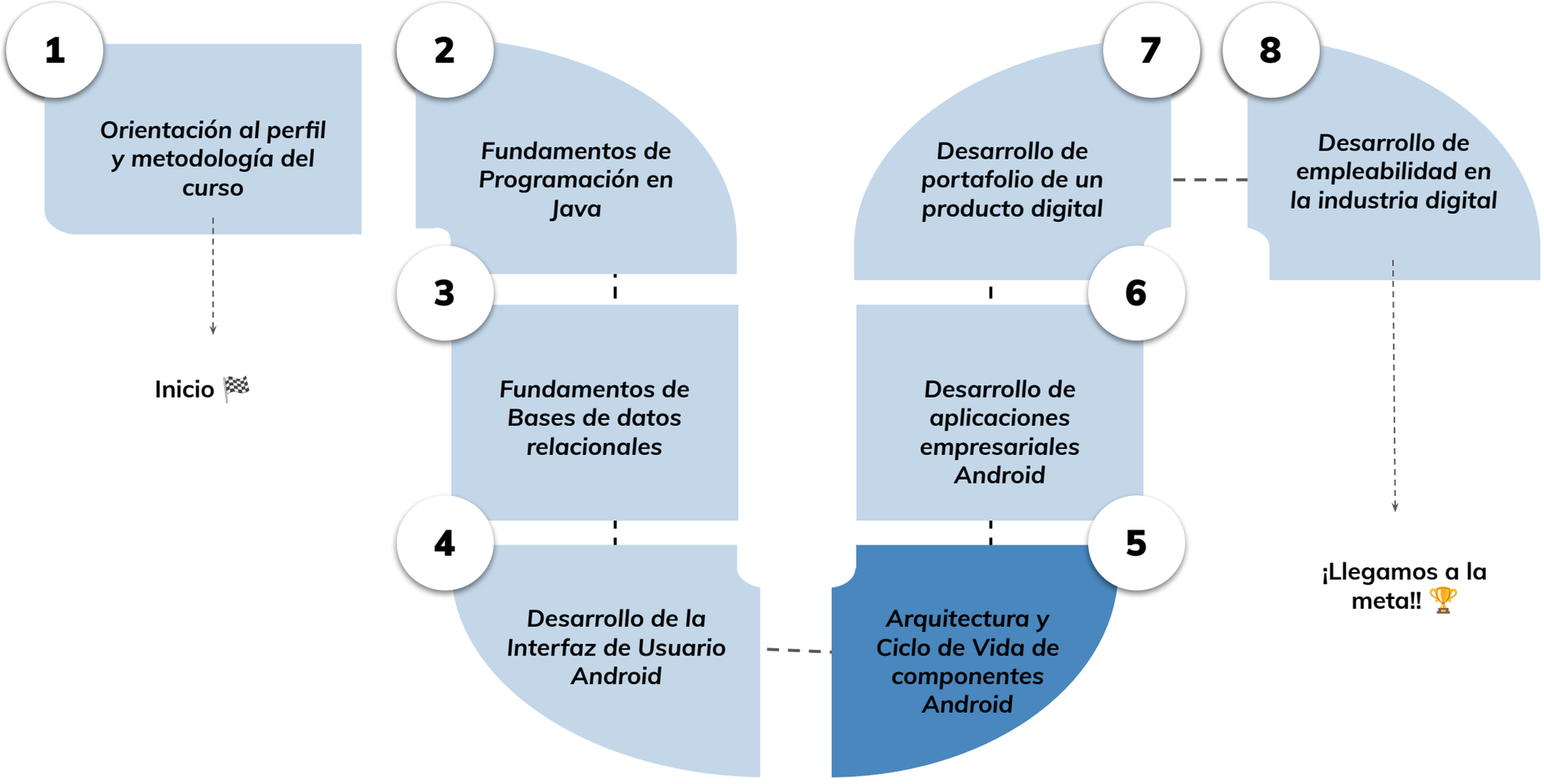
M5 AE1 | Clase El entorno kotlin - Parte I

AE: Reconocer las principales características del lenguaje kotlin para el desarrollo de aplicaciones móviles android nativo.

alkemy

Hoja de ruta

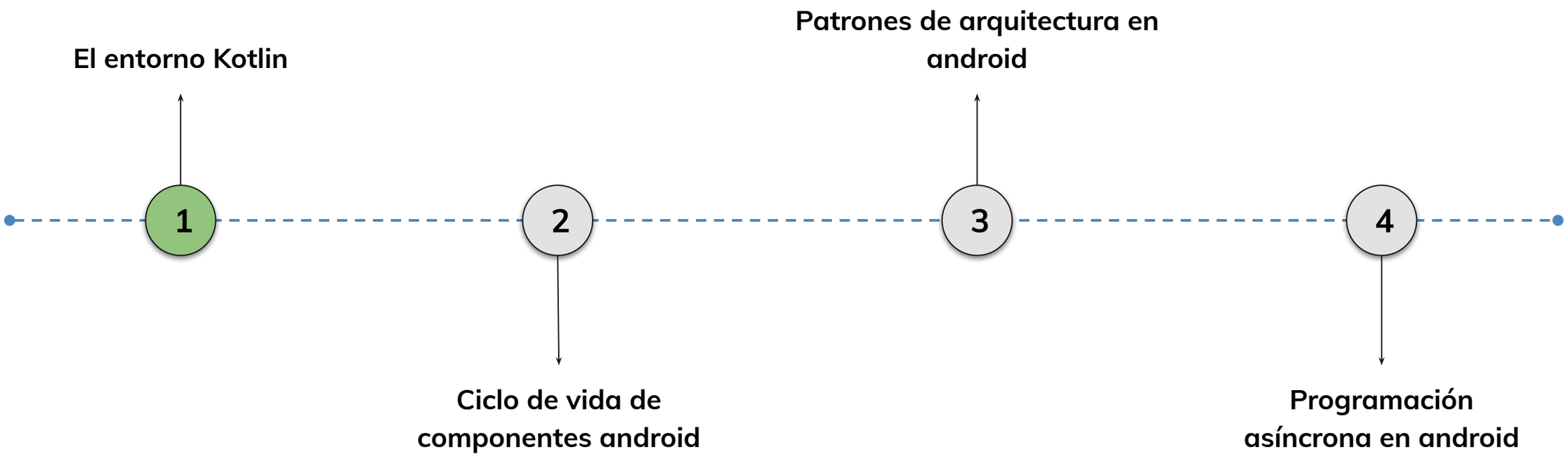
*¿Cuáles **módulos** conforman el programa?*



alkemy

Roadmap de la lección

*¿Cuáles **lecciones** estaremos estudiando en este módulo?*



alkemy

Repaso clase anterior

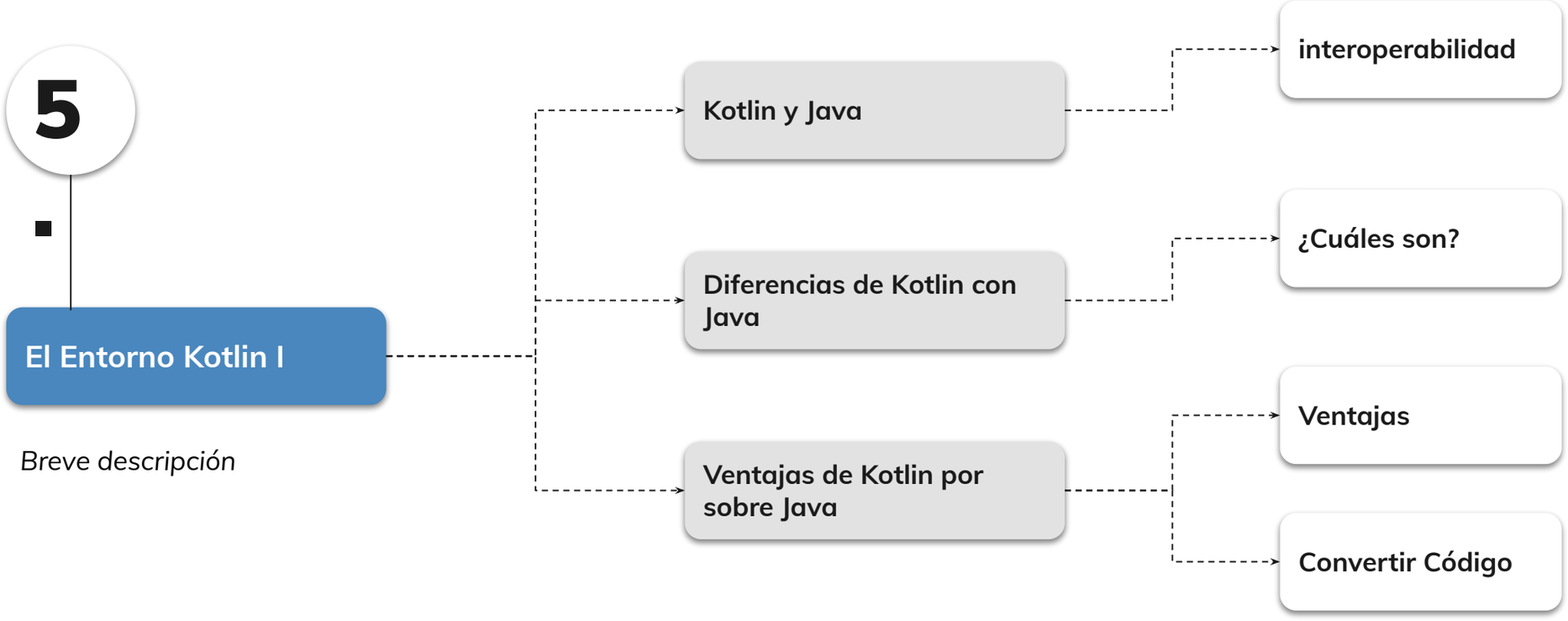
¿Quedó alguna duda?

En la clase anterior trabajamos 📚:

- ✓ *Configurar Git en un entorno de desarrollo.*
- ✓ *Dominar los comandos básicos de Git para crear, clonar y gestionar repositorios.*

Learning Path

¿Cuáles temas trabajaremos hoy?



alkemy

Learning Path

¿Cuáles temas trabajaremos hoy?

5.

El Entorno Kotlin II

- Integrando Kotlin a un proyecto en Android → Métodos
- Lenguaje Kotlin → Características
- Kotlin: Funcional e Imperativo → Funcionamiento
- Los archivos Kotlin → Acitividad N°2

alkemy

Objetivos de aprendizaje

¿Qué aprenderás?

- *Cómo integrar Kotlin a un proyecto en Android*
- *Conocer cuáles son las características del Lenguaje Kotlin*
- *Kotlin: Funcional e Imperativo*
- *Los archivos Kotlin*
- *Conocer sobre Kotlin y Java, y la interoperabilidad*
- *Entender las diferencias de Kotlin con Java*
- *Diferenciar de Kotlin con Java*

› Introducción

Introducción

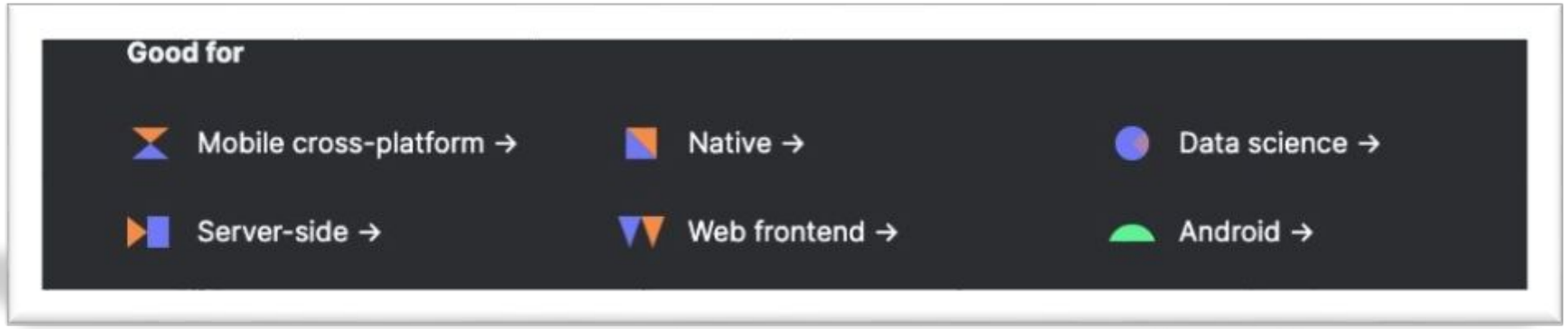
Kotlin es un lenguaje de programación moderno y conciso utilizado oficialmente en el desarrollo de aplicaciones Android. Tiene características poderosas, como la seguridad del nulo para prevenir NullPointerExceptions en tiempo de compilación. También aborda la interoperabilidad con Java, explora conceptos como el contexto, presenta View Binding y ofrece Companion objects para crear miembros estáticos en una clase y compartir funcionalidades comunes.

Introducción

- 2010: Creado por JetBrains
- 2017: Oficialmente soportado por Android.
- 2019: Anunciado por Google como lenguaje principal para Android.
-

Permite trabajar sobre:

- JVM
- JavaScript
- Kotlin Native



¿Porqué Kotlin?

Conciso

Reduce drásticamente la cantidad de Boiler Plate

Seguro

Reduce muchos tipos de errores como los “Null Pointer exceptions”

Interoperable

Puede operar con librerías existentes para la JVM, Android y el navegador.

¿Porqué Kotlin?

- Documentación oficial.

Tool-friendly

Choose any Java IDE or build from the command line

IntelliJ IDEA

Bundled with both IntelliJ IDEA Community Edition and IntelliJ IDEA Ultimate

Android Studio

Bundled with Android Studio

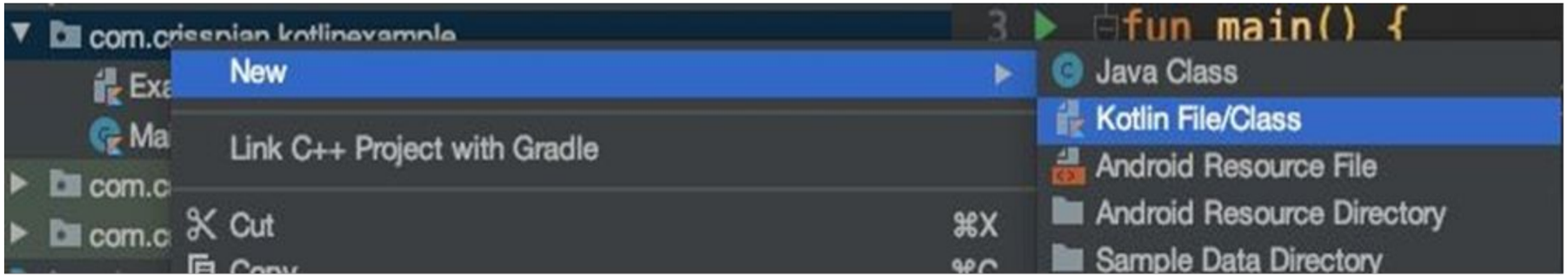
Eclipse

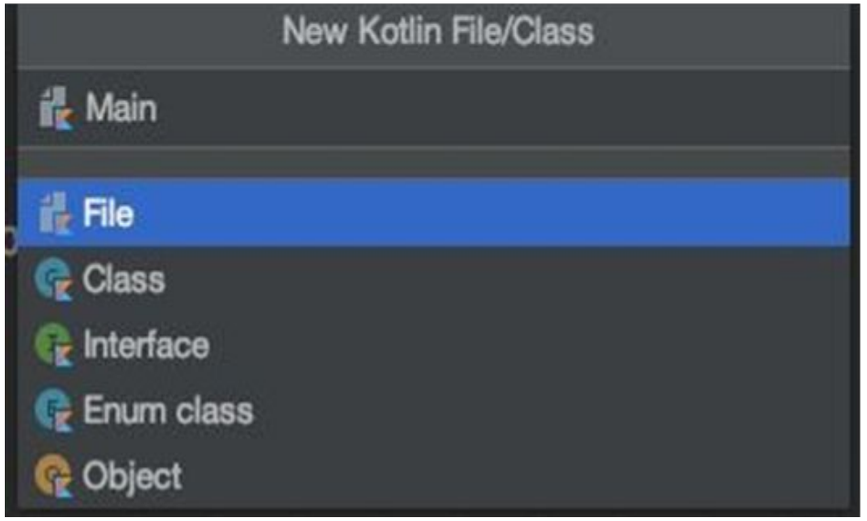
Install the plugin from the Eclipse Marketplace

Standalone Compiler

Use any editor and build from the command line

Hola Mundo kotlin





1. Crear un proyecto en android studio.
2. Crear un nuevo Kotlin File/class

Hola Mundo kotlin

```
package
com.mc.kotlinexample // 1

fun main() {          // 2
  println("Hola mundo
  mundial...")  // 3


  }
```

Kotlin y Java, la interoperabilidad

¿Por qué hablamos de interoperabilidad?

Kotlin ofrece una ventaja clave: la interoperabilidad con Java. Esto permite combinar código de ambos lenguajes en un proyecto sin problemas. Puedes utilizar librerías Java en Kotlin y viceversa, facilitando la migración de proyectos existentes. Importar clases Java en Kotlin es simple, y viceversa. Ahora, veamos ejemplos de "Hola mundo" en ambos lenguajes.

Java:

```java
public class JavaClass {
    public static void javaMethod() {
        System.out.println("Método de Java");
    }
}
```

Kotlin:

```kotlin
fun main() {
    JavaClass.javaMethod()
}
```

› Diferencias de Kotlin con Java

Diferencias de Kotlin con Java

¿Qué es Java?

Java es un lenguaje de programación orientado a objetos de alto nivel con gran popularidad en la industria del software. Destaca por su portabilidad, lo que permite ejecutar código en diferentes sistemas operativos sin modificaciones. Se utiliza en software empresarial, aplicaciones móviles (Android), aplicaciones de escritorio, web y servicios web. Es conocido por su simplicidad, legibilidad y escalabilidad, siendo una elección común para diversos proyectos de desarrollo de software.

Escribir código en java:

```
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hola, mundo!");
    }
}
```

Diferencias de Kotlin con Java

¿Qué es kotlin?

Kotlin es un lenguaje moderno de propósito general que puede compilarse a JavaScript. Desarrollado en 2011 como alternativa más concisa y segura a Java, combina características de varios lenguajes y es interoperable con código Java. Su sintaxis concisa reduce la cantidad de código necesario, y es el lenguaje oficial para el desarrollo de aplicaciones Android, lo que ha impulsado su popularidad en la comunidad móvil.

Escribir código en kotlin:

```
fun main() {
    println("Hola, mundo!")
}
```

Comparación entre kotlin y Java

```kotlin
package com.crisspian.kotlinexample

import ...

class MainActivity : AppCompatActivity() {

    private lateinit var button: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        button = findViewById(R.id.button)
        button.setOnClickListener(object : View.OnClickListener {
            override fun onClick(p0: View?) {
                Toast.makeText( context: this@MainActivity,
                    text: "KOTLIN", Toast.LENGTH_SHORT).show()
            }

        })
    }
}
```

```java
package com.crisspian.kotlinexample;

import ...

public class MainActivity2Java extends AppCompatActivity {

    private Button button;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main_activity2_java);

        button = findViewById(R.id.button);
        button.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View view) {
                Toast.makeText( context: MainActivity2Java.this,
                    text: "JAVA", Toast.LENGTH_SHORT).show();
            }
        });
    }
}
```

Comparación Fragmento

```java
import ...

public class BlankFragmentJava extends Fragment {

    private static final String ARG_PARAM1 = "param1";
    private static final String ARG_PARAM2 = "param2";

    private String mParam1;
    private String mParam2;

    public BlankFragmentJava() {
        // Required empty public constructor
    }

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        if (getArguments() != null) {
            mParam1 = getArguments().getString(ARG_PARAM1);
            mParam2 = getArguments().getString(ARG_PARAM2);
        }
    }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup co
                             Bundle savedInstanceState) {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_blank_java, cont
    }
```

```kotlin
import ...

private const val ARG_PARAM1 = "param1"
private const val ARG_PARAM2 = "param2"

class BlankFragmentKotlin : Fragment() {

    private var param1: String? = null
    private var param2: String? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        arguments?.let { it: Bundle
            param1 = it.getString(ARG_PARAM1)
            param2 = it.getString(ARG_PARAM2)
        }
    }

    override fun onCreateView(
        inflater: LayoutInflater, container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        // Inflate the layout for this fragment
        return inflater.inflate(R.layout.fragment_blank_kotlin, co
    }

    companion object {
        @JvmStatic
        fun newInstance(param1: String, param2: String) =
```

Listeners: lambda o formato tradicional

```
button = findViewById(R.id.button)
button.setOnClickListener(object : View.OnClickListener {
    override fun onClick(p0: View?) {
        Toast.makeText( context: this@MainActivity,
            text: "KOTLIN", Toast.LENGTH_SHORT).show()
    }

})
```

```
button = findViewById(R.id.button)
button.setOnClickListener { it: View!
    Toast.makeText(
        context: this@MainActivity,
        text: "KOTLIN", Toast.LENGTH_SHORT
    ).show()
}
```

› Ventajas de Kotlin por sobre Java

Ventajas de Kotlin por sobre Java

Concisión de código:

Kotlin tiene una sintaxis más concisa que Java, lo que mejora la legibilidad y acelera el desarrollo al permitir escribir la misma funcionalidad en menos líneas de código.

Inferencia de tipos:

Kotlin tiene un sistema de inferencia de tipos avanzado que permite omitir la especificación explícita de tipos en variables y parámetros, ya que el lenguaje puede deducirlos automáticamente según el contexto. Esto simplifica el código al reducir su verbosidad.

alkemy

Ventajas de Kotlin por sobre Java

Nulabilidad segura:

Kotlin aborda el problema de los valores nulos (NullPointerExceptions) al incorporar la nulabilidad en su sistema de tipos. Obliga al desarrollador a manejar explícitamente los valores nulos, reduciendo así los errores y aumentando la seguridad del código.

Funciones de extensión:

Kotlin permite definir funciones de extensión que se pueden añadir a clases existentes sin modificarlas, lo que extiende sus funcionalidades de manera conveniente sin requerir herencia o cambios en el código fuente original.

alkemy

Ventajas de Kotlin por sobre Java

Programación funcional:

Kotlin admite la programación funcional con funciones de orden superior, lambdas y secuencias, lo que facilita un estilo de programación más declarativo y expresivo para manipular y transformar datos.

Interoperabilidad con Java:

Kotlin es completamente interoperable con Java, lo que permite utilizar librerías de Java en proyectos de Kotlin sin problemas, facilitando la migración de proyectos existentes y el intercambio de código entre ambos lenguajes. Su enfoque en la productividad, seguridad y legibilidad del código ha llevado a su creciente popularidad entre los desarrolladores.

alkemy

< > LIVE CODING

Nuestro Hola Mundo Kotlin:

Vamos a crear nuestros primeros pasos en Kotlin.

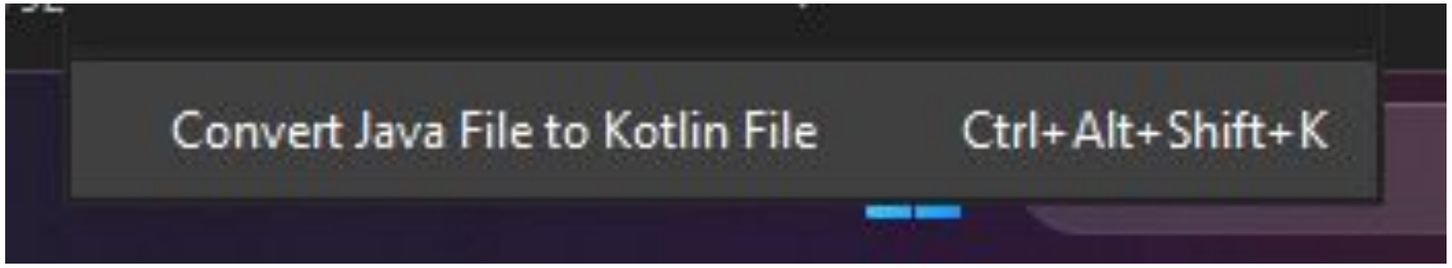
Implementos una simple llamada que imprime por la terminal un “Hola mundo” al ser ejecutada, para eso, crear una función Main que tenga el código dentro y al ser ejecutado imprime en la terminal

Tiempo: 15 min

Convertir codigo

¿Cómo?:

Android Studio nos permite convertir nuestro proyectos o alguna clase en particular que deseemos de Java a Kotlin usando solamente la opción “Convert Java File to Kotlin File” en el menu de “Code”



< > LIVE CODING

Convertir una clase Java a Kotlin:

Crear una clase de datos Java y pasarla a Kotlin usando la opción de convertir código mostrada anteriormente

1. *Crear una Clase Usuario*
2. *Que tenga un constructor*
3. *Que requiera nombre y mail*
4. *Que tenga setters y getters*
5. *Convertir la clase a Kotlin*
6. *¿Qué cambio?*

Tiempo: 20 min

```
*
*
*
*
*
* * * * *
```

```
public class LSketcher {

public void draw() {  int rows = 5;
int cols = 5

for (int row = 0; row < rows; row++) {  println("*");
if (row == rows - 1) {
    for (int col = 0; col < cols; col++) {  print("* ");
  }
}
}
}private void print(String text) {  System.out.print(text);
}
private void println(String text) {
System.out.println(text);
}
}
```

```
*     *
*     *
*     *
*     *
*     *
* * * * *
```

```
public class USketcher {

  public void draw() {  int rows = 5;
  int cols = 5;

 ......
}
 private void print(String text) {
 System.out.print(text);
}
 private void println(String text) {
 System.out.println(text);
}
}
```

alkemy

¿Alguna consulta?

alkemy

Momento:

Time-out!

⏰5 -10 min.

alkemy

› Integrando Kotlin a un proyecto en Android

Integrando Kotlin a un proyecto en Android

Para integrar Kotlin en un proyecto Android, tenemos que seguir los siguientes pasos:

Configuración del entorno de desarrollo:

Asegúrate de tener el entorno de desarrollo adecuado para Android, como Android Studio, instalado en tu máquina. Android Studio proporciona soporte nativo para Kotlin.

Crear un nuevo proyecto:

Abre Android Studio y crea un nuevo proyecto de Android como lo harías normalmente.

Integrando Kotlin a un proyecto en Android

Configurar el soporte de Kotlin:

Durante la creación de un proyecto en Android Studio, elige "Kotlin" como lenguaje preferido. Si ya tienes un proyecto en Java, puedes agregar Kotlin a él siguiendo pasos adicionales.

Convertir archivos a Kotlin:

Para usar Kotlin en tu proyecto, convierte archivos individuales de Java a Kotlin haciendo clic derecho en el archivo Java en Android Studio, selecciona "Convert Java File to Kotlin File" y sigue las instrucciones.

Integrando Kotlin a un proyecto en Android

Agregar la dependencia de Kotlin:

En el archivo build.gradle (nivel de módulo) de tu proyecto, agrega la dependencia de Kotlin en la sección de dependencias. Debes agregar la siguiente línea:

```
implementation "org.jetbrains.kotlin:kotlin-stdlib:$kotlin_version"
```

Asegúrate de reemplazar $kotlin_version con la versión actual de Kotlin.

Sincronizar el proyecto:

Después de agregar la dependencia de Kotlin, sincroniza los cambios haciendo clic en "Sync Now" en Android Studio para descargar las bibliotecas necesarias. Luego, podrás escribir y utilizar código Kotlin en tu proyecto Android, ya sea creando nuevas clases en Kotlin o convirtiendo gradualmente las clases Java existentes a Kotlin.

› Características del Lenguaje Kotlin

Caracteristicas de Kotlin

Interoperabilidad con Java: Kotlin es 100% compatible con Java, lo que permite usar bibliotecas y frameworks de Java en proyectos Kotlin sin problemas.
Sintaxis concisa: Kotlin tiene una sintaxis más simple y compacta que Java, lo que reduce la cantidad de código repetitivo y hace el código más legible.
Null Safety: Kotlin tiene un sistema de tipos que ayuda a evitar errores comunes de nulidad (como el NullPointerException), proporcionando una mayor seguridad en el manejo de valores nulos.
Funciones de primera clase: Permite tratar funciones como objetos, lo que facilita la programación funcional. Soporta funciones de extensión y lambdas.
Compatibilidad multiplataforma: Kotlin no solo se usa en Android, sino también para desarrollo en la web (Kotlin/JS), en el backend (Kotlin/JVM), y en aplicaciones de escritorio (Kotlin/Native).
Soporte para programación funcional y orientada a objetos: Kotlin admite ambos paradigmas de programación, lo que lo hace versátil y flexible.
Coroutines: Kotlin ofrece corutinas para simplificar la programación asincrónica, facilitando la gestión de tareas en segundo plano y mejorando el rendimiento.
Compilación rápida: Kotlin tiene un compilador eficiente, lo que hace que el tiempo de compilación sea más rápido que en muchos otros lenguajes.

Características del Lenguaje Kotlin

Seguridad del nulo:

En Kotlin, como en muchos otros lenguajes de programación, el NullPointerException (NPE) es una excepción que ocurre cuando intentas acceder a un objeto que no ha sido inicializado o que tiene un valor nulo (null). El NPE es una de las excepciones más comunes en la programación y puede llevar a comportamientos inesperados o errores en tu aplicación si no se maneja adecuadamente.

En Kotlin, todas las variables se consideran no nulas de manera predeterminada, lo que significa que no pueden contener valores nulos a menos que se declare explícitamente que pueden hacerlo. Esto ayuda a prevenir NPEs en tiempo de compilación, lo que significa que el compilador te advertirá si estás intentando acceder a un valor nulo sin manejarlo adecuadamente.

alkemy

Características del Lenguaje Kotlin

Inferencia de tipos:

Kotlin cuenta con un sistema de inferencia de tipos que permite omitir declaraciones explícitas de tipos de variables, ya que el compilador puede deducirlos automáticamente según el contexto. Esto mejora la concisión y legibilidad del código Kotlin.

Funciones de extensión:

Kotlin permite agregar funciones a clases existentes sin cambiar su código original mediante funciones de extensión, lo que amplía sus capacidades y mejora la legibilidad del código.

alkemy

Características del Lenguaje Kotlin

Programación funcional:

Kotlin cuenta con un sistema de inferencia de tipos que permite omitir declaraciones explícitas de tipos de variables, ya que el compilador puede deducirlos automáticamente según el contexto. Esto mejora la concisión y legibilidad del código Kotlin.

Funciones de extensión:

Kotlin permite agregar funciones a clases existentes sin cambiar su código original mediante funciones de extensión, lo que amplía sus capacidades y mejora la legibilidad del código.

alkemy

Características del Lenguaje Kotlin

Clases de datos:

Kotlin permite definir clases de datos de manera concisa, generando automáticamente métodos comunes como getters, setters, equals(), hashCode() y toString(). Esto simplifica la creación de clases para el almacenamiento de datos.

Smart casts:

Kotlin puede realizar "smart casts" en contextos específicos, lo que permite utilizar un tipo más específico sin necesidad de una conversión explícita.

alkemy

› Kotlin: Funcional e Imperativo

Kotlin: Funcional e Imperativo

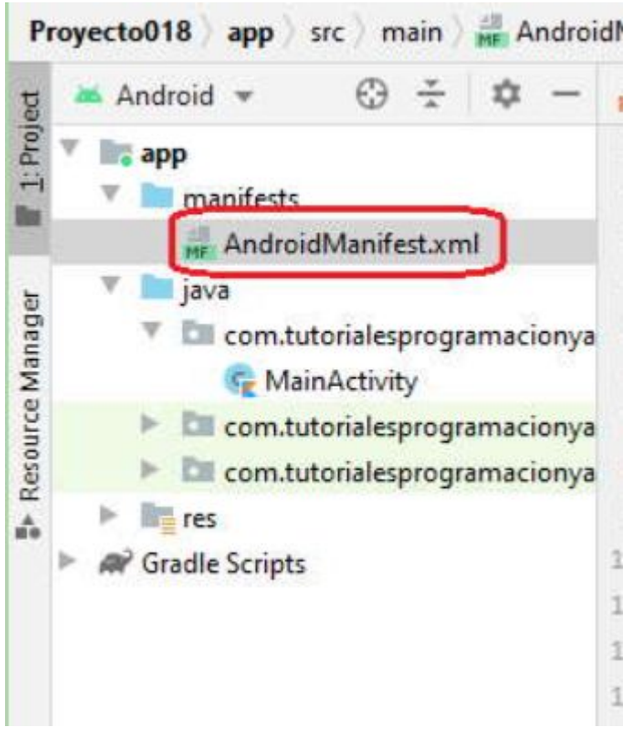
Kotlin es un lenguaje versátil que combina programación funcional e imperativa. Ofrece características como funciones de orden superior, lambdas y secuencias para programación funcional, que enfatiza "qué" hacer en lugar de "cómo". También es imperativo, lo que permite controlar el flujo y realizar operaciones mutables. Los desarrolladores pueden elegir el enfoque según la necesidad, lo que hace a Kotlin adecuado para una variedad de escenarios de desarrollo.

› Los archivos Kotlin

Los archivos Kotlin

En Kotlin, los archivos tienen extensión ".kt" y contienen el código fuente de los programas. Pueden contener clases, interfaces, funciones y otros elementos de código. Las estructuras comunes en un archivo Kotlin incluyen declaraciones de paquete, importaciones, declaraciones de funciones y declaraciones de clases o interfaces.

En un archivo Kotlin,vamos a encontrar las siguientes estructuras:



Los archivos Kotlin

Clases:

En Kotlin, las clases se definen con la palabra clave "class" y sirven como plantillas que describen propiedades y métodos que definen un objeto. Estas clases permiten crear instancias para acceder a sus propiedades y métodos.

Funciones:

En Kotlin, las funciones se definen con "fun" y son bloques de código reutilizables para realizar tareas específicas. Pueden estar dentro de una clase o a nivel de archivo.

alkemy

Los archivos Kotlin

Propiedades:

En Kotlin, las propiedades se definen con "val" o "var" y son variables asociadas a una clase u objeto. Las "val" son de solo lectura (inmutables), mientras que las "var" son mutables.

Estructuras de control:

En Kotlin, puedes usar estructuras de control como condicionales (if-else, when) y bucles (for, while) para gestionar el flujo de ejecución de tu código.

alkemy

Los archivos Kotlin

Clases de datos:

Puedes usar la palabra clave "data" para crear clases de datos. Estas clases generan automáticamente métodos comunes como getters, setters, equals(), hashCode() y toString(), lo que las hace adecuadas para el almacenamiento y manipulación de datos.

Extensiones:

En Kotlin, las funciones de extensión permiten agregar funcionalidad extra a clases existentes sin modificar su código original. Esto amplía las capacidades de las clases sin requerir herencia o cambios en el código fuente original.

alkemy

Los archivos Kotlin

```
fun main() {
    // Declaración de variables
    val nombre = "Juan"  // val -> variable inmutable
    var edad = 25        // var -> variable mutable

    // Estructura if
    if (edad >= 18) {
        println("$nombre es mayor de edad.")
    } else {
        println("$nombre es menor de edad.")
    }

    // Estructura repetitiva for
    for (i in 1..5) {
        println("Número: $i")
    }

    // Estructura repetitiva while
    var contador = 0
    while (contador < 3) {
        println("Contador: $contador")
        contador++  // Incrementa el contador
    }
}
```

alkemy

< > LIVE CODING

Agregar Kotlin a un proyect:

Crear un proyecto Java o usar uno ya existente y agregar la compatibilidad con Kotlin agregando las dependencias correspondientes y validar que tenemos la última versión de Kotlin corriendo

Tiempo: 15 min

¿Alguna consulta?

Resumen

¿Qué logramos en esta clase?

- ✓ *Interoperabilidad entre Java y Kotlin*
- ✓ *Diferencias*
- ✓ *Ventajas de Kotlin*
- ✓ *Integrando Kotlin a un proyecto en Android*
- ✓ *Características del Lenguaje Kotlin*
- ✓ *Kotlin: Funcional e Imperativo*
- ✓ *Los archivos Kotlin*

< ¡Muchas gracias! >

alkemy